

DATA-260 Lab 1 – Report

Pair 06 · Uday Patel (Part A) and Sreeramachandra Sai Achutuni (Parts B and C)

Item	Value
Repository	https://github.com/UdayPate/data260-pair06 (private)
Tagged commit	_____ (to be filled in at the end: final commit hash, tag lab1)
Hardware	Lenovo Legion Slim 5 (16-inch, Ryzen 5 7640HS, 16 GB RAM, RTX 4060 8 GB), Windows, MySQL 8 local
Local model	qwen3:8b via Ollama (Parts B and C)

Section 1 – Goals

Lab 1 asks for a Handshake-style career platform with two personas, **Student** and **Company**, and a hand-built AI assistant that works on the platform’s own data. The platform uses React (front end), FastAPI (back end) and MySQL (database). The assistant uses a local Ollama model and an agent loop written by hand, with no agent framework. The lab is worth 40 points.

Part	What it asks for	Points	Owner
A	The platform: both personas, REST API, validation, authorization, Swagger or Postman documentation, seed generator	22	Uday
B	Agent loop, one model client class, five tools, five transcripts	12	Partner
C	Confirmation gate, prompt-injection defence, tool-call reliability table	6	Partner

This report covers all three parts. Sections 1 to 6 are Part A. Sections 7 to 10 are Parts B and C.

The goals for Part A, taken from the handout:

- **Student:** sign up (bcrypt) and sign in/out with a token; profile with every updatable field and a picture; search jobs by company or title and filter by category and city; job and company details; apply with a PDF resume and filter own applications by Pending, Reviewed or Declined; events in date order, search, register when eligible; browse students by name or college and filter by major; AI chat on the home page.
- **Company:** sign up and sign in/out; post jobs; see own postings and who applied; open a student profile, preview the PDF resume and set the status; company profile with picture; Students tab (name, college, skills); Events tab (post events, see registered students).
- **Back end and front end:** validate every input, encrypt passwords, handle exceptions, restrict users to authorized actions; React with Bootstrap, Axios, loading and error states, separate components, pages and services; Swagger UI or a Postman collection.
- **Data:** at least 300 jobs, 1,500 applications, 200 students, 50 companies and 100 events across CITY_SET, generated with `random_state = SEED` by a committed script.

How we know each goal is met: the requirement-to-code-to-test table is in `docs/PART_A_CHECKLIST.md`, the API evidence is in section 6, and the test counts are in section 6 and `METRICS.md`.

Section 2 – System design

2.1 – Architecture

```
Browser (React 18 + Vite + react-bootstrap, http://localhost:9061)
  | pages -> services/*.js -> services/api.js (Axios, adds the token, one place for errors)
  v HTTP + JSON, header Authorization: Bearer <JWT>
FastAPI (http://localhost:9060, Swagger UI at /docs)
  routers/ HTTP only: path, status code, which role may call it
  schemas/ Pydantic v2: validates every request body and shapes every response
  services/ business rules and queries (jobs, events, students, preferences, applications)
  models.py SQLAlchemy 2.0 tables
  | ^
  | +-- services/ is also what the AI assistant tools call (Parts B and C),
  v so the assistant gets the same permission rules as the website
MySQL 8 (database p06_handshake) uploads/ (resumes: private; profile_pics: public)
```

JWT = JSON Web Token. CORS (Cross-Origin Resource Sharing) allows only the React origin (port 9061).

2.2 – Layers and why they are split

Layer	Folder	Rule it follows
Router	backend/app/routers/	Thin. Reads the request, calls one service function, returns the result.
Schema	backend/app/schemas/	Every request body is a Pydantic model that checks types, lengths and ranges. The job, event, preference, profile-update, status-update and chat bodies also use <code>extra="forbid"</code> , so an unknown field such as <code>company_id</code> gets 422. The signup and login bodies do not forbid extra fields; unknown fields there are ignored and never reach the database.
Service	backend/app/services/	All business rules and SQL. The website and the assistant call the same functions.
Model	backend/app/models.py	Table definitions, enums, UNIQUE constraints.
Core	backend/app/core/	Password hashing, JWT, role checks (<code>deps.py</code>), safe uploads.

Front end: pages never call Axios directly. Pages call `services/*.js`, which call `services/api.js`. The `useAsync` hook gives every request a loading state, an error state and a reload. Array filters are sent as repeated keys (`paramsSerializer: { indexes: null }`), which is what FastAPI expects.

2.3 – Data model

Twelve tables (all in `backend/app/models.py`):

Table	Purpose	Notable constraints
students	Student account and profile	email unique, password_hash (bcrypt)
student_skills, student_experience	Skills and work history	skill unique per student; cascade delete
student_preferences	Saved job and event preferences (read by the assistant)	one row per student
companies	Company account and profile	email unique
jobs, job_skills	Postings and required skills	enums for category and pay period; skill unique per job
applications	A student applying to a job, with resume path and status	UNIQUE (student_id, job_id)
events, event_eligible_majors	Events and the majors allowed (one row All = open to everyone)	UNIQUE (event_id, major)
event_registrations	A student registered for an event	UNIQUE (student_id, event_id)
saved_items	Jobs or events saved by the assistant's save_job_or_event tool	UNIQUE (student_id, item_type, item_id)

In this report a **job posting** is a row in `jobs` (the API paths say `/jobs`). Relationships: a company has many job postings and events; a job has many applications; a student has many applications and registrations. Foreign keys use `ON DELETE CASCADE`. Application status is the enum Pending, Reviewed, Declined; job category is the enum `full_time`, `part_time`, `on_campus`, `internship`.

2.4 – Authentication

- Passwords are hashed with `bcrypt` (the library's default cost, 12 rounds; `core/security.py`). At least 8 characters, one letter and one digit, at most 72 bytes (the `bcrypt` limit). The hash is never returned by any endpoint.
- `POST /auth/login` returns a JWT (HS256, 8 hours) that holds the user id, the role and the expiry. The client sends it as `Authorization: Bearer <token>`. A wrong email and a wrong password give the same reply (`test_login_wrong_password_and_unknown_email_look_identical`).
- `core/deps.py` turns the token into the current user. A missing, forged or expired token gives **401**. `require_student` and `require_company` give **403** to the wrong role.
- The JWT secret and the database password come from `backend/.env`, which is git-ignored.

2.5 – Authorization and role rules

Rule	Behaviour	Test that checks it
Missing, garbage, expired, wrongly signed token, or deleted account	401	<code>test_auth.py::test_missing_or_garbage_token_is_401</code> , <code>test_expired_token_is_401</code> , <code>test_token_signed_with_wrong_secret_is_401</code> , <code>test_token_for_deleted_account_is_401</code>
A user's own data	Always taken from the token (<code>/students/me</code>), never from an id in the URL	By design (the routes have no id); no single test is named for it
Another person's resume	404 , not 403 , for another student, for a company that does not own the job, and for an id that does not exist (same reply body); 401 when not logged in	<code>test_applications.py::test_everyone_else_gets_404_for_a_resume</code>
Another company's application	GET <code>/applications/{id}</code> gives 404 ; a student calling that company-only endpoint gets 403	<code>test_applications.py::test_applicant_detail_privacy</code>
Applicants list of a job posting the company does not own	403 (the posting itself is public, so its existence is not secret); a student gets 403 ; a missing posting is 404	<code>test_applications.py::test_applicants_list_is_private_to_the_job_owner</code>
Company editing a job posting or event it does not own	403	<code>test_jobs.py::test_cannot_edit_someone_elses_job_or_a_missing_job</code> , <code>test_events.py::test_cannot_edit_someone_elses_event_or_a_missing_one</code>
Event registration when the student's major is not eligible (or the student has no major and the event is not open to all)	403 with the reason in the message	<code>test_events.py::test_ineligible_student_is_refused_with_a_clear_reason</code> , <code>test_student_without_a_major_can_only_join_open_events</code>
Registering for a past event	409	<code>test_events.py::test_cannot_register_for_past_or_missing_events_or_as_other_roles</code>
Duplicate application, duplicate registration, signup with an existing email	409 (the UNIQUE constraint is the final guard against two clicks at once)	<code>test_applications.py::test_cannot_apply_twice_but_can_apply_to_other_jobs</code> , <code>test_events.py::test_cannot_register_twice_but_others_can</code> , <code>test_auth.py::test_duplicate_email_is_rejected_case_insensitively</code>

Rule	Behaviour	Test that checks it
Resumes	Private. Served only by GET /applications/{id}/resume to the applicant and the owning company, with Cache-Control: private, no-store ; the resumes folder is not mounted as static files	test_resume_files_are_not_served_as_static_files, test_profiles.py::test_resumes_folder_is_not_publicly_served
Profile pictures	Public under /media/profile_pics/	test_profiles.py::test_upload_picture_and_serve_it
What a company sees of a student in the list (GET /students)	Name, college, major, degree, graduation year, skills, city, state, email and GPA ; no phone, no date of birth	test_directory.py::test_company_list_shows_contact_and_academic_fields_but_never_dob_phone_or_password
What a company sees of one student (GET /students/{id}, applicant detail)	The list fields plus country, career objective, experience and phone ; never the date of birth	test_directory.py::test_company_opens_a_full_student_profile_without_dob_or_password, test_applications.py::test_applicant_detail_has_full_profile_without_date_of_birth
What a student sees of a peer	Name, college, major, degree, graduation year, skills, picture, career objective, experience; no email, phone, GPA, date of birth, city or state	test_directory.py::test_peers_never_get_email_phone_gpa_dob_or_city

Uploads are checked by content: resumes must start with %PDF- (5 MB limit); pictures must be PNG, JPEG or WebP by magic bytes (2 MB limit). File names are generated by the server.

2.6 – Where the assistant plugs in

POST /assistant/chat (students only) takes {message, conversation_id} and returns {reply, conversation_id, tool_calls: [{name, arguments, result}]}. The router, schemas, tests and the React chat window exist. Parts B and C replace only the body of run_assistant() in backend/app/services/assistant.py. The student comes from the token, so the assistant can only act for the person who is logged in.

2.7 – Home-page activity heatmap, themes and avatars

The student home page shows a heatmap of applications per day, fed by GET /students/me/activity?days=365 (students only: a company gets 403, no token gets 401; days is 1 to 730). The server counts the student’s own applications by the date they were made and returns only the days that have at least one.

The front end has two role-based looks. The data-theme attribute on the <html> element is **student** (arcade look: dark green background, coral accents, pixel font for titles, navigation, buttons and badges, square corners, hard offset shadows) or **company** (serious look: the same colours,

IBM Plex Sans only, 6 px corners, 1 px borders, flat shadows, coral only on the main button and the active navigation item). It follows the logged-in role, and on the sign-in and sign-up pages it follows the Student/Company switch. One stylesheet, `frontend/src/theme.css`, holds both themes as CSS variables.

Profile pictures use a generated fallback: a pixel-art face for students and a tile with the initials for companies, both chosen from the name so the same name always gives the same picture. A real uploaded picture always wins; the generated one is shown only when there is none or it fails to load.

Section 3 – Pair parameters

Parameter	Rule from the handout	Value
PAIR	Assigned on Canvas	06
PORT_BASE	$9000 + (\text{PAIR} \times 10)$	9060
Back end (FastAPI)	PORT_BASE	9060
Front end (React dev server)	PORT_BASE + 1	9061
Database and Kafka prefix	p{PAIR}	p06 (database p06_handshake)
SEED	PAIR, for every generator and <code>random_state</code>	6 (<code>random.Random(6)</code> , <code>Faker.seed_instance(6)</code>)
CITY_SET	Assigned on Canvas	San Jose, Sunnyvale, Mountain View

Source: `backend/app/config.py` (the same values are in `README.md`). Seeded demo logins: `demo.student@sjsu.edu` and `demo.company@example.com`; the password is not repeated here.

Section 4 – Seed counts

The generator is `backend/seed/generate_seed.py`. It is run with `python -m seed.generate_seed --reset`, which drops and rebuilds `p06_handshake`. Every random choice uses `random.Random(6)` or a `Faker` seeded with 6, so running it again gives identical data. Cities for jobs, events, students and companies are chosen from `CITY_SET`.

Counts below are the real MySQL row counts measured by `python -m scripts.collect_metrics` on 2026-10-08 23:17 (`docs/part_a_metrics.md`, `METRICS.md`):

Table	Rows	Required by the handout	Met
Companies	50	50	yes
Students	200	200	yes
Job postings	300	300	yes
Applications	1500	1500	yes
Events	100	100	yes
Event registrations	400	n/a	n/a

Table	Rows	Required by the handout	Met
Saved preferences	151	n/a	n/a
Saved items	150	n/a	n/a
Student skills	877	n/a	n/a
Job skills	1073	n/a	n/a

Rows beyond the handout’s minimums exist so every feature has something to show. The first six jobs are fixed “scenario” jobs (three remote Data Science jobs and one Data Analyst job in each of the three cities) with deadlines in 2027, so the Part B test queries always find them. Evidence of the run: `RUN_LOG.txt` (seed output) and the row-count screenshot in section 5.

Section 5 – Screenshots

Screenshots are in `docs/screenshots/`. Only Figure 31 (git history) is still pending: it needs both partners’ commits.

5.1 – App screens (React)



Figure 1: Student home page: application activity heatmap and the AI assistant card (the chat box continues below the part captured)

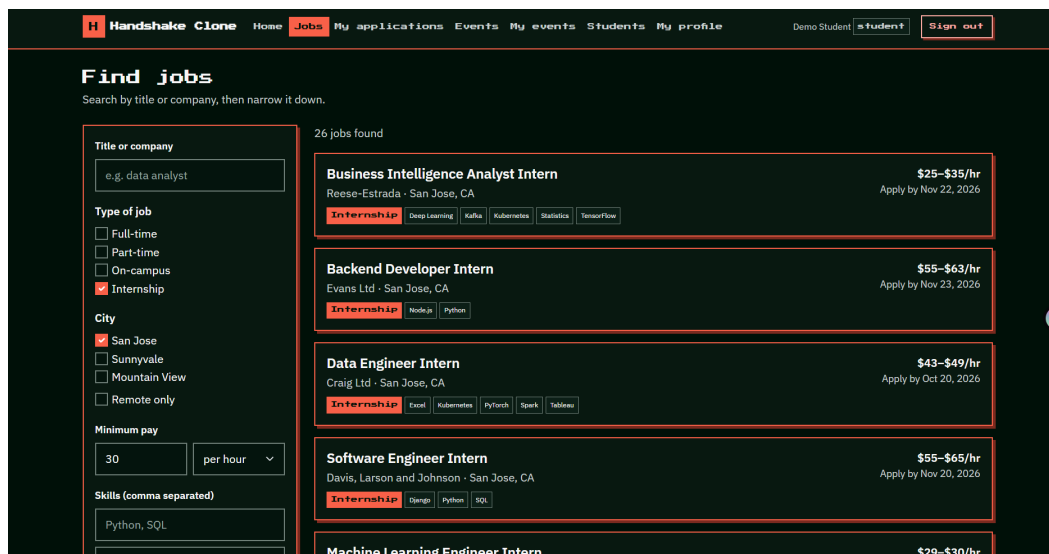


Figure 2: Job search with filters (Internship, San Jose, minimum pay 30 per hour: 26 jobs found)

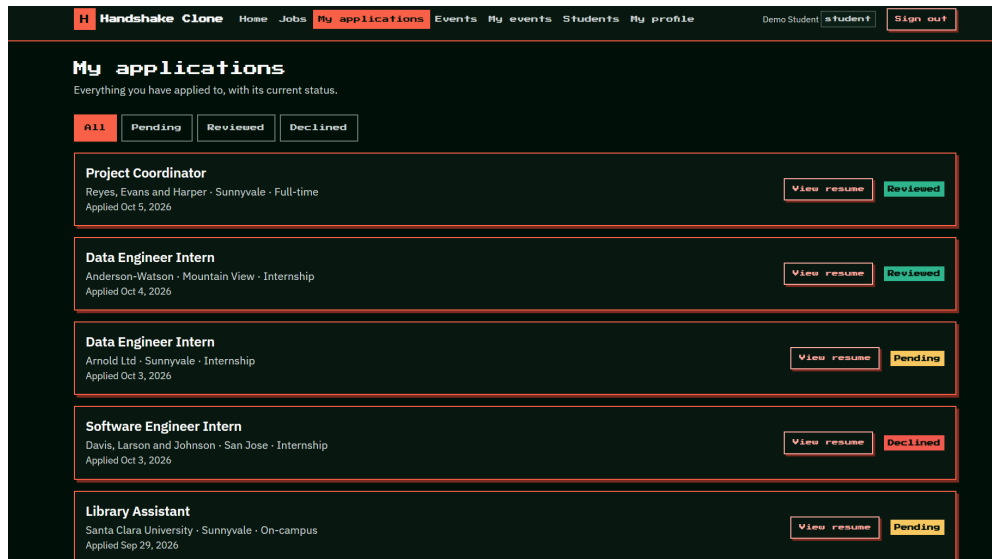


Figure 3: My applications with the status filters and a View resume button on every row (the resume window itself is not open in this capture)

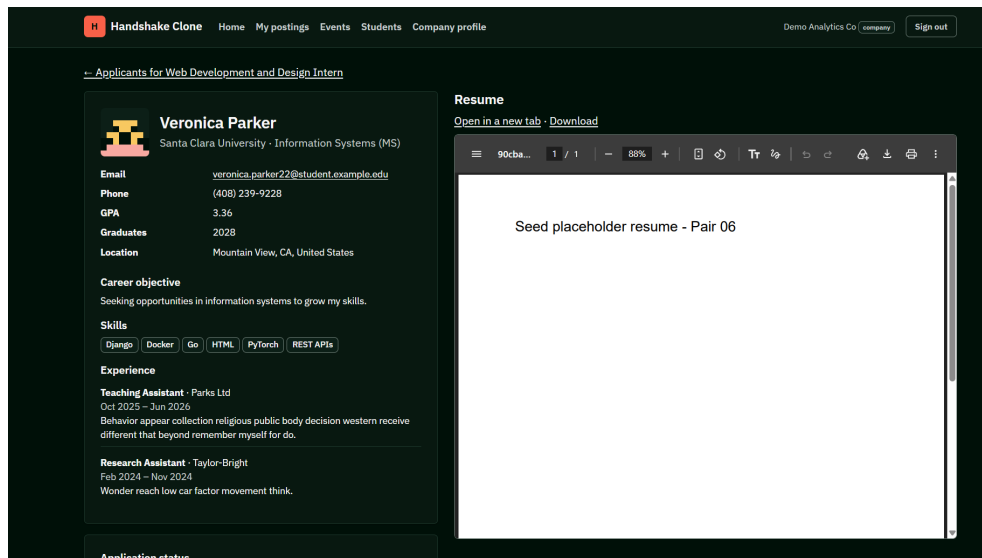


Figure 4: Company view of one applicant: email, phone and GPA but no date of birth, skills, experience and the resume preview (the status buttons are below the part captured)

5.2 – Swagger UI: sign-in, profile and jobs (student token)

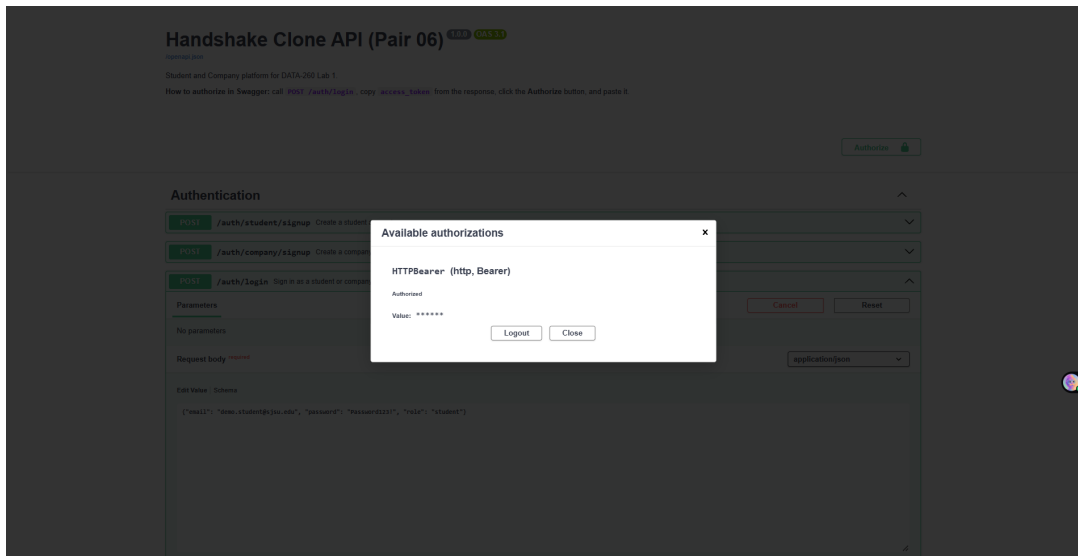


Figure 5: Swagger Authorize dialog with the login request body (token hidden)

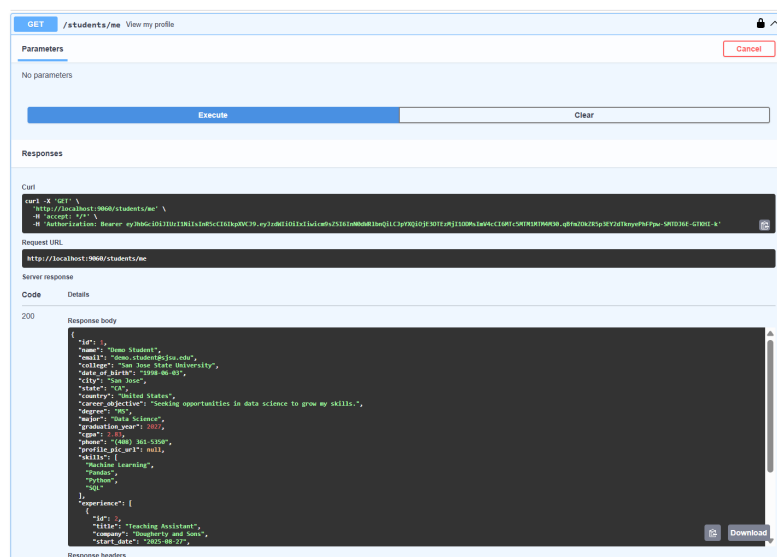


Figure 6: GET /students/me for the demo student (own view)

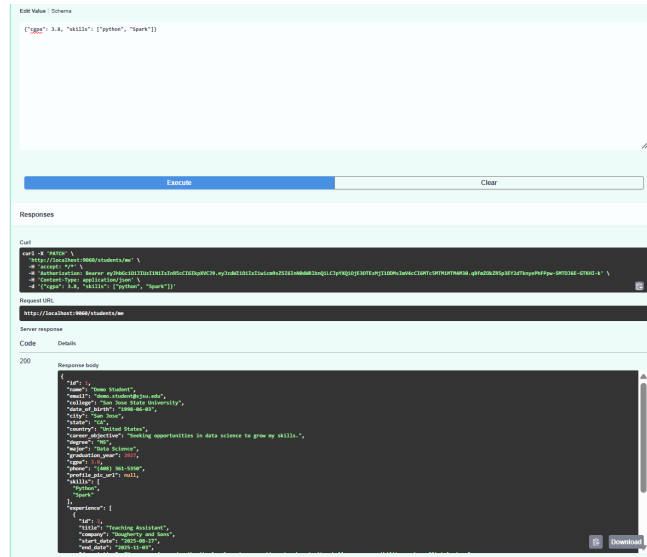


Figure 7: PATCH `/students/me` changes GPA and skills

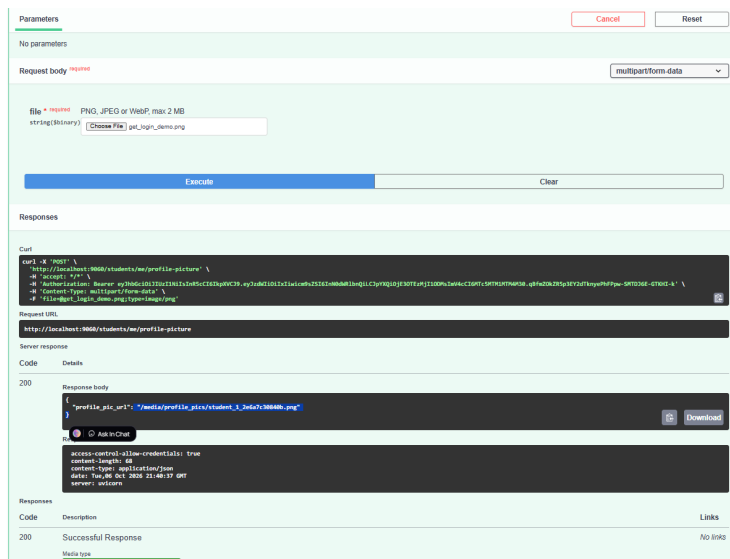


Figure 8: POST `/students/me/profile-picture` returns the public picture URL

5.3 – Swagger UI: applications and events (student token)

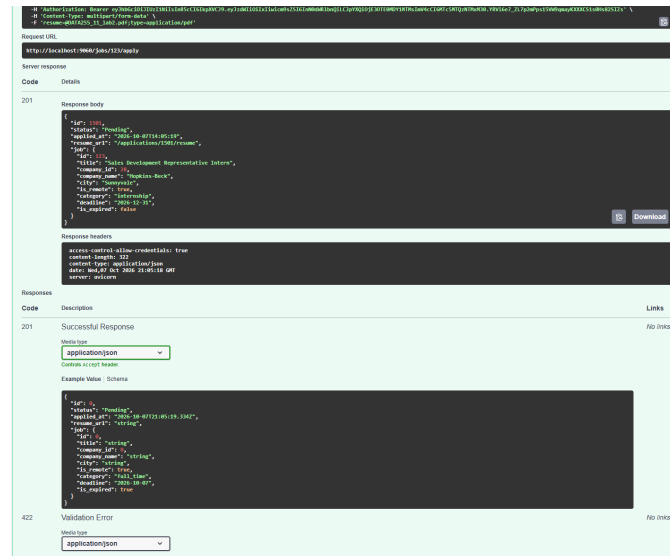


Figure 11: POST `/jobs/123/apply` with a PDF resume returns 201

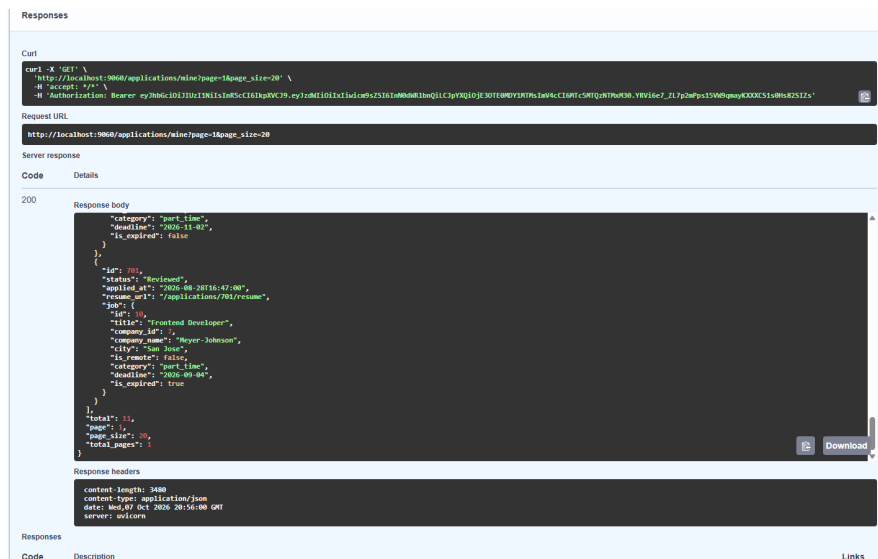


Figure 12: GET `/applications/mine`

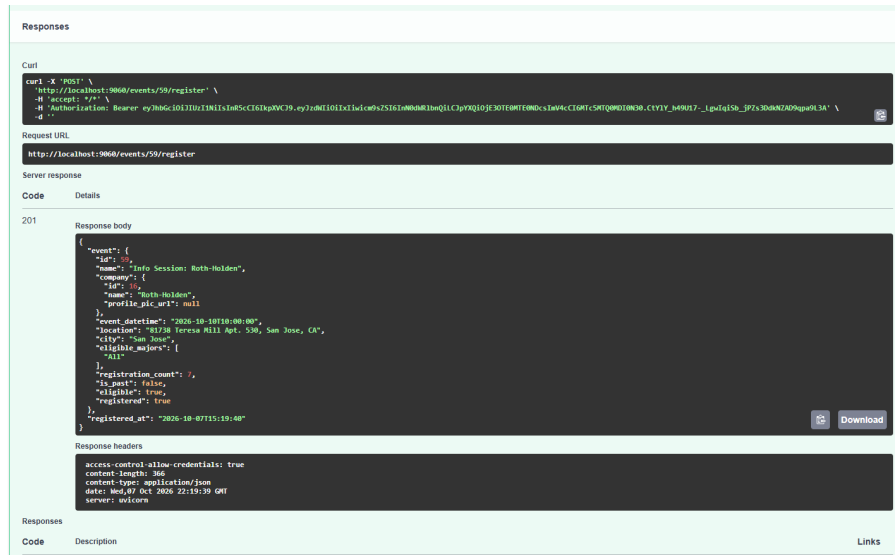


Figure 15: POST /events/59/register returns 201



Figure 16: Registering again returns 409

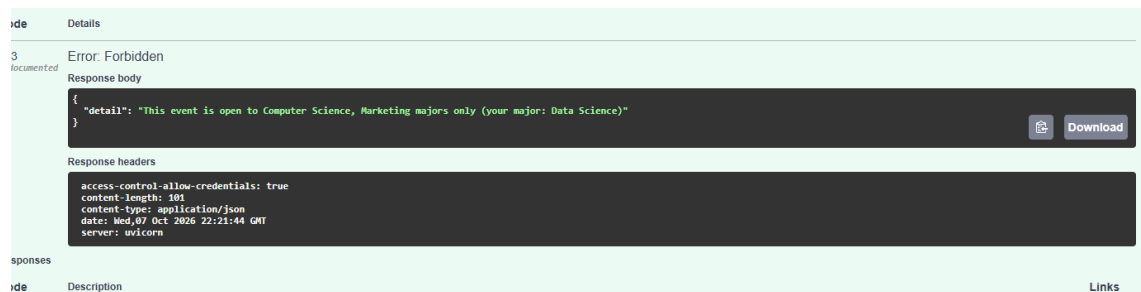


Figure 17: Registering for an event closed to the student's major returns 403

5.4 – Swagger UI: student directory, student token and company token

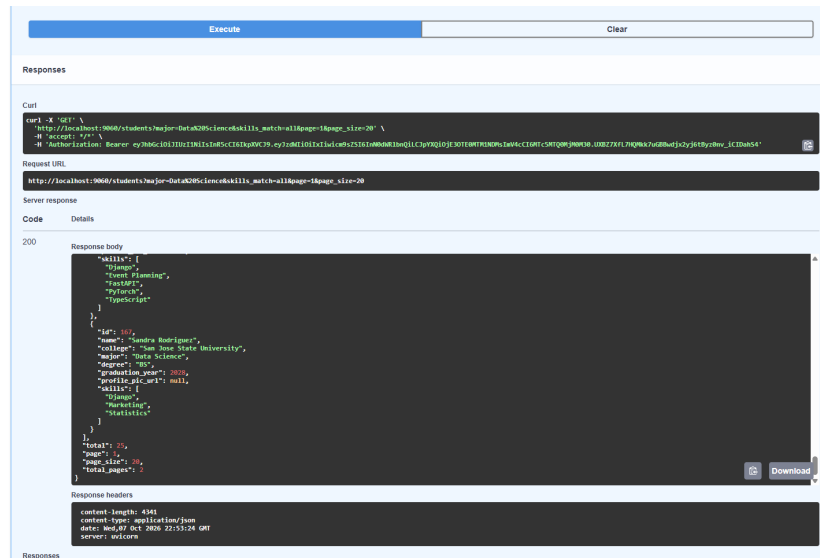


Figure 18: GET /students as a student (peer view, no email)

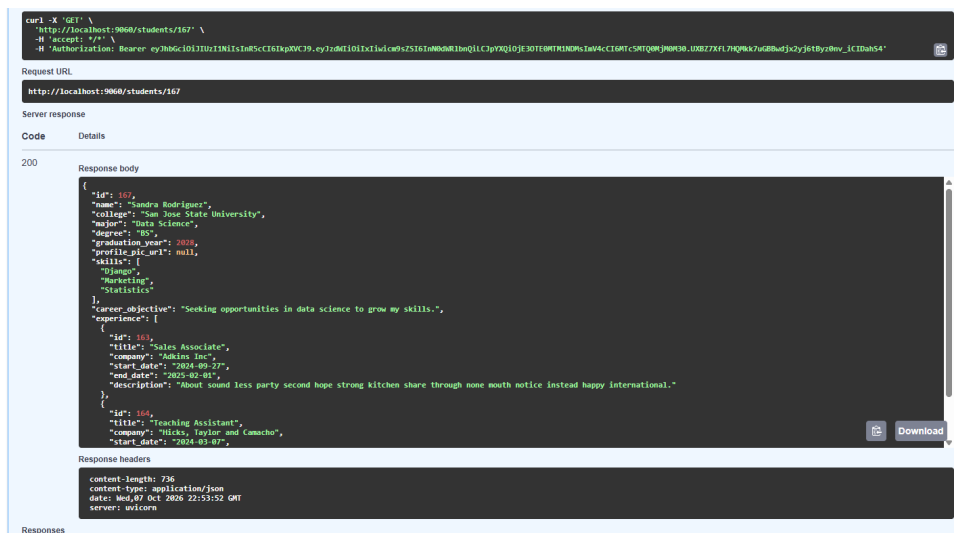


Figure 19: GET /students/167 as a student (peer profile)

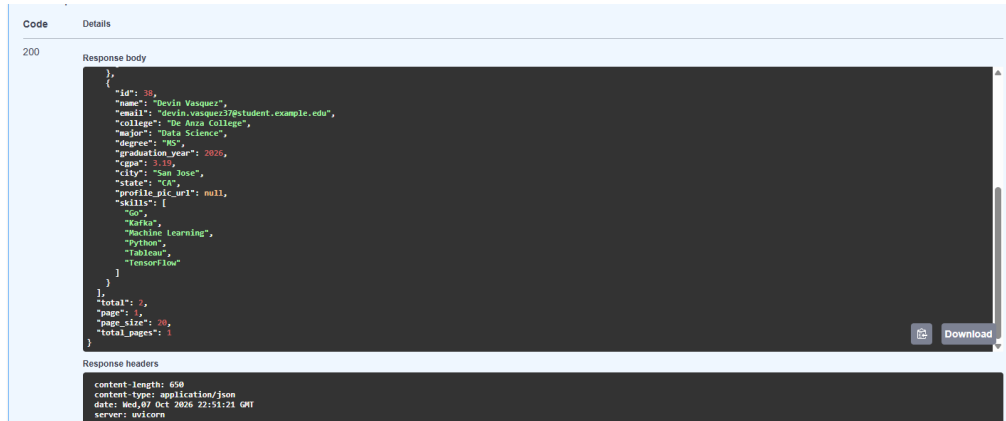


Figure 20: GET /students as a company (email and GPA shown)

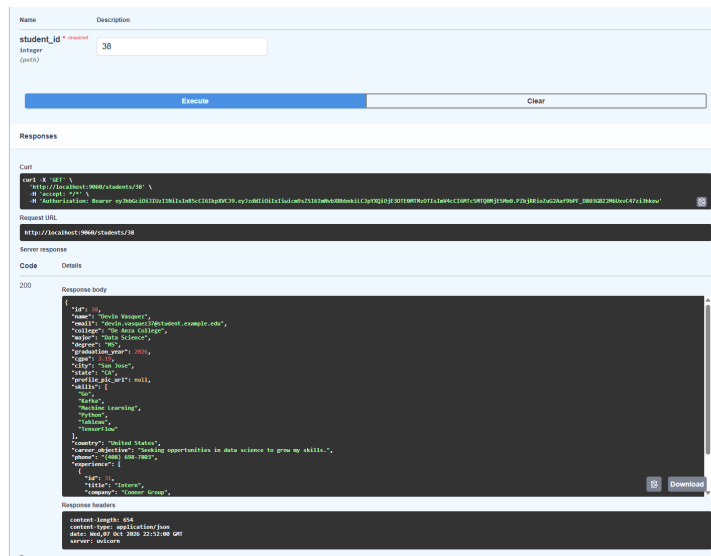


Figure 21: GET /students/38 as a company (email, GPA and phone)

5.5 – Swagger UI: company token

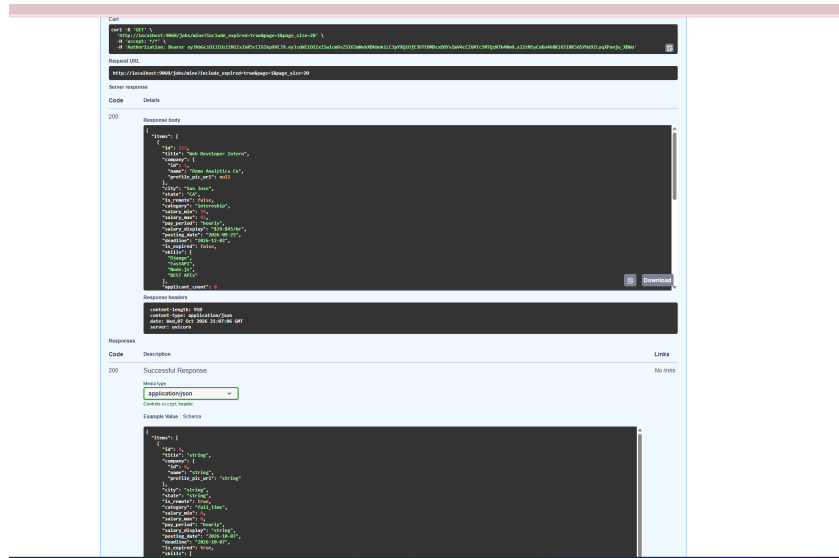


Figure 22: GET /jobs/mine

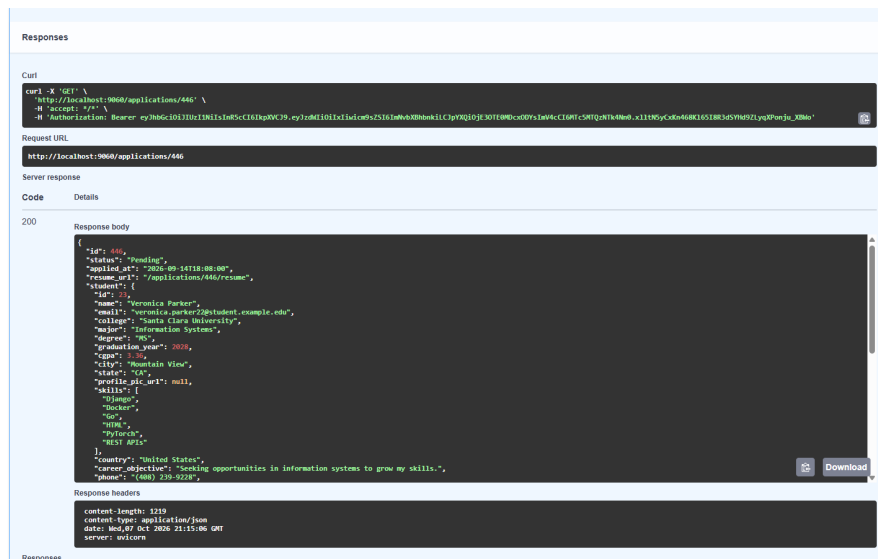


Figure 23: GET /applications/446 (applicant detail)



Figure 24: PATCH /applications/446/status sets Reviewed

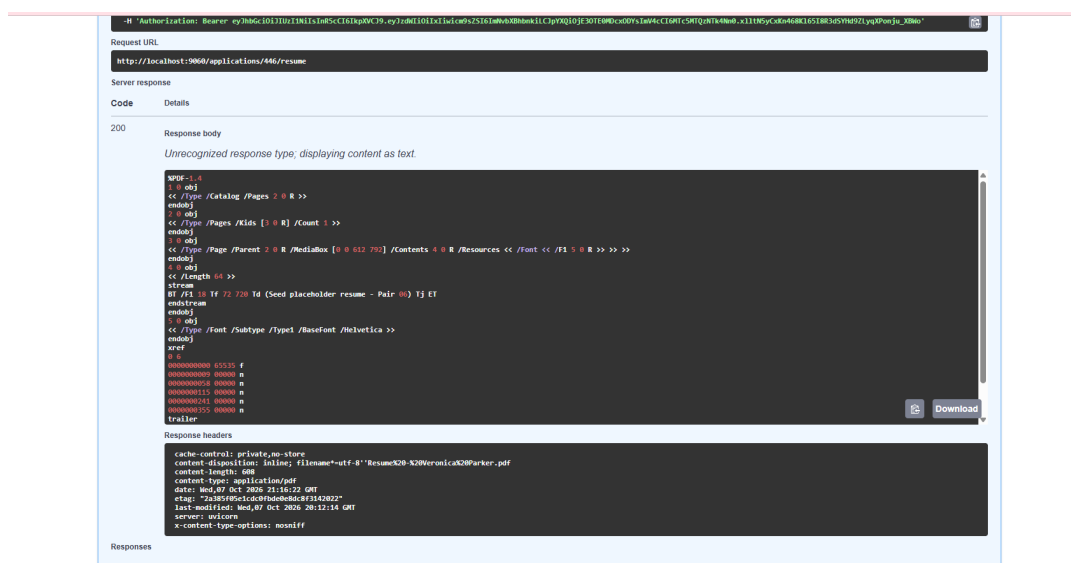


Figure 25: GET /applications/446/resume returns the PDF with private headers

Request URL		
http://localhost:3000/events/mine?include_past=true&page=1&page_size=20		
Server response		
Code	Details	
200	Response body <pre>{ "items": [{ "id": 1, "name": "Resume Workshop", "company": { "id": 1, "name": "Demo Analytics Co", "profile_pic_url": null }, "event_datetime": "2020-09-20T13:30:00", "location": "712 Nicole Crossroad Apt. 230, Mountain View, CA", "city": "Mountain View", "eligible_majors": ["All"], "registration_count": 0, "is_past": true, "eligible": null, "registered": null }], "total": 1, "page": 1, "page_size": 20, "total_pages": 1 }</pre> Download Response headers <pre>content-length: 382 content-type: application/json date: Wed, 07 Oct 2020 22:23:09 GMT server: unicorn</pre>	
Responses		
Code	Description	Links
200	Successful Response	No links

Figure 26: GET /events/mine

Server response		
Code	Details	
201	Response body <pre>{ "id": 101, "name": "Data Engineering Aectup", "company": { "id": 1, "name": "Demo Analytics Co", "location": "San Jose", "state": "CA", "industry": "Financial Services", "description": "Machine energy a production. Whole same floor against major cap their. Much behind nor record rock production particular. Such friend behavior song. Knowledge behind cultur a play fact.", "contact_email": "demo.company@example.com", "contact_phone": "(408) 683-5723", "website": "https://www.demo.example.com", "profile_pic_url": null }, "event_datetime": "2020-11-12T18:30:00", "location": "180 Main St, San Jose", "city": "San Jose", "eligible_majors": ["Data Science"], "registration_count": 0, "is_past": false, "eligible": null, "registered": null, "description": "Pipelines, Kafka and Spark in practice.", "can_register": null, }</pre> Download Response headers <pre>access-control-allow-credentials: true content-length: 884 content-type: application/json date: Wed, 07 Oct 2020 22:23:48 GMT server: unicorn</pre>	
Responses		

Figure 27: POST /events returns 201

5.6 – Tests, row counts and git history

```
• (venv) PS C:\Users\patel\data260-pair06\backend> python -m p
.....
=====
venv\Lib\site-packages\starlette\testclient.py:40
  C:\Users\patel\data260-pair06\backend\venv\Lib\site-packag
    _PortalFactoryType = typing.Callable[[], typing.ContextM

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-wa
363 passed, 1 warning in 180.89s (0:03:00)
○ (venv) PS C:\Users\patel\data260-pair06\backend> █
```

Figure 28: pytest summary (363 passed)

```
Test Files 17 passed (17)
Tests 210 passed (210)
Start at 22:11:35
Duration 15.47s (transform 1.36s, setup 45.22s, collect 4.23s, tests 53.66s, environment 34.15s, prepare 4.21s)
○ (venv) PS C:\Users\patel\data260-pair06\frontend> █
```

Figure 29: npm test summary (210 passed in 17 test files)

```
• (venv) PS C:\Users\patel\data260-pair06> Get-Content docs\row_counts.sql | & "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u root -p --table
Enter password: *****
+-----+
| table | rows |
+-----+
| Companies | 50 |
| Students | 200 |
| Job postings | 300 |
| Applications | 1500 |
| Events | 100 |
| Event registrations | 400 |
| Saved preferences | 151 |
| Saved items | 150 |
| Student skills | 877 |
| Job skills | 1073 |
+-----+
○ (venv) PS C:\Users\patel\data260-pair06> █
```

Figure 30: MySQL row counts for p06_handshake (the SQL is in docs/row_counts.sql)

SCREENSHOT PENDING

git_01_history.png

Figure 31: Git commit history showing both partners (taken last)

A note on dates: the Swagger screenshots were taken on 6 to 7 October 2026 against a live database. Two of them (api_04, application id 1501, and api_20, event id 101) created extra rows, so the live database held 1,501 applications and 101 events at that time. The row counts in section 4

were measured later, after `python -m seed.generate_seed --reset` (RUN_LOG start 2026-10-08 22:54), and show 1,500 and 100.

Section 6 – API tests

6.1 – Swagger evidence

Swagger UI (<http://localhost:9060/docs>) was used with the demo student and demo company logins. Each line says what the screenshot in section 5 proves. The Swagger set has no screenshot of a 401 or a 404; those two codes are covered by the pytest output in 6.2.

Screenshot	What it proves
api_01_authorize_dialog.png	Bearer-token authentication is wired into Swagger: POST /auth/login takes {email, password, role} and the token goes into Authorize (value masked).
api_02_jobs_filters_200.png	200. Search and filters work together: q=data science, category=internship, is_remote=true, skills=Python&skills=Machine Learning (repeated keys), skills_match=any, include_expired=false; total is 3 with paging fields.
api_03_student_preferences_200.png	200. A student reads their own saved preferences (categories, cities, roles, minimum hourly rate, remote, event interests). This is the data the assistant's get_student_preferences tool reads.
api_04_student_apply_201.png	201. Multipart PDF upload creates an application with status Pending; the resume is exposed only as the protected path /applications/1501/resume.
api_05_student_my_applications_200.png	200. A student lists only their own applications (total 11) with date, status (here Reviewed) and job summary.
api_06_student_events_200.png	200. Upcoming events (include_past=false) with registration_count, eligible and registered flags for the logged-in student.
api_07_student_events_filtered_200.png	200. Event filters by date_from, date_to and major; 18 results. The last one shows registered: true.
api_08_event_register_201.png	201. Eligible student registers for event 59 (open to All).
api_09_event_register_409_duplicate.png	409 Conflict, You are already registered for this event (duplicate registration rule).
api_10_event_register_403_major.png	403 Forbidden, eligibility rule: the event is open to Computer Science and Marketing only and the student's major is Data Science.
api_11_student_directory_peer_no_email.png	200. GET /students?major=Data Science as a student: peer cards contain name, college, major, degree, graduation year, picture and skills, and no email, phone or GPA (total 25).

Screenshot	What it proves
api_12_student_profile_peer_no_email.png	200. A student opening a peer profile gets the limited view plus career objective and experience; still no email, phone, GPA or date of birth.
api_13_company_students_with_email.png	200. The same directory called with a company token includes <code>email</code> and <code>cgpa</code> .
api_14_company_student_detail_with_email_phone.png	200. A company opening one student sees email, GPA and phone. The date of birth is not in the visible part of the body; its absence is checked by <code>pytest</code> (see 6.2).
api_15_company_jobs_mine_200.png	200. A company lists only its own job postings, with <code>applicant_count</code> (expired ones included because <code>include_expired=true</code>).
api_16_company_application_detail_200.png	200. Company opens applicant 446: status, resume URL and the student's profile with contact details.
api_17_company_status_reviewed_200.png	200. PATCH <code>/applications/446/status</code> changes Pending to Reviewed.
api_18_company_resume_pdf_200_private.png	200. The resume is served only through the authenticated route: <code>content-type: application/pdf</code> , <code>cache-control: private</code> , <code>no-store</code> , <code>content-disposition: inline</code> , <code>x-content-type-options: nosniff</code> . The file is the seed's placeholder PDF.
api_19_company_events_mine_200.png	200. A company lists only its own events (here one past event with 8 registrations).
api_20_company_post_event_201.png	201. Company posts an event (id 101) restricted to Data Science; the response echoes the company and eligible majors.

6.2 – Automated tests (pytest)

Command: `python -m pytest -q` in `backend/`, on an in-memory SQLite database (never MySQL). The summary line of the final run (Figure 28) is:

```
363 passed, 1 warning in 180.89s (0:03:00)
```

An earlier run of the same 363 tests is saved in `docs/part_a_tests_after_heatmap.txt` (192.49 s). Before the activity endpoint was added the suite had 346 tests; that older run is in `docs/part_a_tests.txt` and `RUN_LOG.txt`. Tests per file (counted with `pytest --collect-only -q`, which lists 363 tests in total):

File	Tests passed	Covers
<code>test_activity.py</code>	17	The application activity heatmap endpoint (per-day counts, own data only, window limits, 401, 403, 422)
<code>test_applications.py</code>	47	Apply, resume rules, applicants, status, private resume route
<code>test_assistant.py</code>	12	The POST <code>/assistant/chat</code> contract (placeholder reply, validation, student-only)

File	Tests passed	Covers
test_auth.py	22	Signup, login, JWT checks, password rules
test_directory.py	26	Student directory and the two role views
test_events.py	72	Events, eligibility, registration, ownership
test_jobs.py	68	Job postings, search and filters, ownership
test_preferences.py	33	Saved preferences
test_profiles.py	66	Profiles, pictures, email changes, public and private files
Total	363	

The role-rule tests behind section 2.5 (each one passed in this run; the test names are in the table in 2.5):

Rule	Status code	Test file and function
No token, garbage token, expired token, wrong secret, deleted account	401	test_auth.py::test_missing_or_garbage_token_is_401, test_expired_token_is_401, test_token_signed_with_wrong_secret_is_401, test_token_for_deleted_account_is_401
Student calling a company-only endpoint, or the reverse	403	test_applications.py::test_applicant_detail_privacy, test_jobs.py::test_cannot_edit_someone_elses_job_or_a_missing_job
Another company's job posting or applicants	403	test_applications.py::test_applicants_list_is_private_to_the_job_owner
Another person's resume or application	404	test_applications.py::test_everyone_else_gets_404_for_a_resume, test_applicant_detail_privacy
Ineligible event registration	403	test_events.py::test_ineligible_student_is_refused_with_a_clear_reason
Duplicate application, registration or email	409	test_cannot_apply_twice_but_can_apply_to_other_jobs, test_cannot_register_twice_but_others_can, test_duplicate_email_is_rejected_case_insensitively
Bad input is refused	422	test_applications.py::test_invalid_status_rejected[extra-field] (unknown field), test_jobs.py::test_invalid_edits_rejected_and_change_nothing, test_events.py::test_invalid_event_rejected[...]
Resume must really be a PDF, at most 5 MB	4xx	test_non_pdf_uploads_rejected[...] (6 cases), test_resume_over_5mb_rejected
Company sees email and GPA but not phone or date of birth in the list; peers see neither	200	test_directory.py::test_company_list_shows_contact_and_academic_fields_but_never_dob_phone_or_password, test_peers_never_get_email_phone_gpa_dob_or_city

Heatmap endpoint: GET /students/me/activity?days=365 returns the logged-in student's appli-

cations per day (students only: 401 without a token, 403 for a company, 422 for days outside 1 to 730), checked by the 17 tests in `test_activity.py`.

6.3 – Front-end tests and API documentation

`npm test` (Vitest, Testing Library, axios-mock-adapter) reported 17 test files and 210 tests passed in 15.47 s (Figure 29). API documentation exports are `docs/openapi.json` and `docs/postman_collection.json`.

Section 7 – Five transcripts

[To be added by Parts B and C from real runs.]

Section 8 – Reliability table

[To be added by Parts B and C from real runs.]

Section 9 – Injection before and after

[To be added by Parts B and C from real runs.]

Section 10 – Assistant tool-call evidence

[To be added by Parts B and C from real runs.]